

RE-MASTERS (RM.Sc.) IN COMPUTER SCIENCE
HWP LABS, BENIN CITY, EDO STATE
PROSPECTUS 2018/2019 SESSION

PREFACE

Objective

RMSC is a programme developed by HWP Labs to equip M.Sc. in Computer Science students with industry-relevant skills in Advance Software Engineering. At the end of the programme, students should be able to create full-stack web applications using JavaScript as well as apply general concepts of programming to other information software development projects-- regardless of the programming language of choice.

Scope

The RMSC SlingshotJS framework (CSC819) covers selected topics in Computer Science such as; Advanced Data Structures, Artificial Intelligence (AI), Compiler Design, Computer Communications and Networking, Database Management System, Design and Analysis of Algorithms, Expert Systems, Human-Computer Interaction (HCI), Information Technology (IT), Operations Research, Pattern Recognition, Programming Languages, Software Engineering and Systems Analysis and Design.

Modules

- ❖ Concepts of programming with JavaScript (ECMAScript 6).
- ❖ Source code editing with Visual Studio Code.
- ❖ Advanced Data Structures with JavaScript Object Notation (JSON).
- ❖ Parallel Processing with Asynchronous JavaScript and XML (AJAX).
- ❖ Server-side Scripting with Node.js Run-time Environment.
- ❖ Document-oriented Database Management with MongoDB.
- ❖ Software Development Life Cycle (SDLC) using Waterfall Model and MVC.
- ❖ Software Prototyping with Hypertext Markup Language (HTML5).

Prerequisite

- ✓ Text Editor –Visual Studio Code [\[download\]](#)
- ✓ Web Browser – Google Chrome [\[download\]](#)
- ✓ JavaScript Run-time Environment - Node.js [\[download\]](#)
- ✓ Database Management System – MongoDB [\[download\]](#)

NOTE: All software packages are currently available for transfer via CD or USB.

COURSE OUTLINE

MODULE 1: Concepts of programming with JavaScript (ECMAScript 6)			
#	LECTURE	SCOPE	EXERCISE
1	Program Structure <u>Components of a Webpage</u> Tags Attributes Elements Shell [] Website Architecture	Programming Domains The Internet World Wide Web	alert.js “Hello, World!”
2	Standard I/O <u>Syntax and Semantics</u> Expressions and Statements Identifiers and Scopes Programming Conventions [] Basic Flowchart of a Computer Program [] Algorithm for Program Design using MVC	Human-Computer Interaction (HCI) Command-line Interface (CLI) Algorithms Formula Translation	prompt.js confirm.js console.js ... loan_calc.js pay_slip.js
3	<u>Operators</u> Arithmetic Operators Assignment Operators Comparison Operators Logical Operators Operator Precedence ($z = -b + a * c / f + j$) ? Unary, Binary and Ternary Operators	Euclidean Division Algebraic Equations Flip Bit Mutation Product-Moment Correlation	modulus.js exponent.js ternary.js ... precedence.js
4	<u>Data Types</u> Primitive Data Types Composite Data Types Abstract Data Types Type Checking (typeof x) Type Conversion (Type Casting)	Data Validation	typeof.js
5	<u>Data Structures</u> Stack Queue *Pointer &Alias Dictionary (Hash Map or Assoc. Array) ? Difference between Stacks and Queues ? Underflow and Overflow ? Applications of Genetic Algorithms	Abstract Data Types Enumerated Type Number Parity Arithmetic Crossover (GA)	array.js enum.js dictionary.js record.js ... jamb.js

6	Control Structures I <u>Decision Statements (Branch)</u> If Statement Switch Statement	Grading Systems Tower of Hanoi Algorithm Game of Life Algorithm Inference Engine (A.I.)	if.js switch.js ... pop_quiz.js
7	Control Structures II <u>Iteration Statements (Loop)</u> For Loop While Loop Do-While Loop For-in Loop Nested Loops Counter-controlled Loops	Data Authentication Multiplication Table Arbitrary Matrices Linear Search Selection Sort Fibonacci Algorithm	for.js while.js do_while.js for_in.js ... series.js nested_loop.js matrix.js tree.js
8	<u>Subprograms</u> Functions and Procedures Recursion Subroutines Parameters and Arguments Lambda Function Nested Subprograms Overloaded Subprograms Generics	Macroinstructions (Macros) Factorial Mean, Median, Mode Expert System (A.I.)	function.js recursion.js lambda.js subroutines.js
9	Abstract Data Types <u>Attributes of Object Oriented Programming</u> Class Members Properties Constructor Methods (Setter, Getter, Logic, Display) Instance of a Class Object of a Class <u>Concepts of Object Oriented Programming</u> Abstraction Encapsulation Inheritance Polymorphism Interface	Data Abstraction Static Properties and Methods Entity Relationship Model Class and Object Diagrams (UML) Automated Teller Machine (ATM)	encapsulation.js inheritance.js
10	<u>Built-in APIs</u> String Number	Application Programming Interface (API) String Buffer and String Builder Anagrams and Palindromes	table.js arguments.js timestamp.js

	Array Math Date Console <u>User-defined APIs (macros/*)</u> Buffer Binary Calendar MariaDB	Virtual Assistant (A.I.)	... salute.js palindrome.js anagram.js
11	Regular Expression ? Applications of Pattern Matching ? Data Sterilization v Data Authentication	Data Sterilization Pattern Recognition (Brute-Force) Character Encoding Place-value Notation	regex.js pattern.js encoding.js ... binary.js hexadecimal.js
12	Errors and Debugging <u>Types of Errors</u> Compile-time Errors Run-time Errors <u>Source of Errors</u> Syntax Errors Semantic Errors Logical Errors ? Common Sources of Errors (Vitamin A)	Software Testing	exit.js debugger.js
13	<u>Exception Handling</u> NumberFormatException ArithmeticException ArrayIndexOutOfBoundsException ClassNotFoundException FileNotFoundException	Software Quality Control	exception.js

MODULE 2: Source code editing with Visual Studio Code

#	LECTURE	SCOPE	EXERCISE
14	Workspace Environment Menu Bar Tabbed Bar Toolbox Navigation Pane Text-editor Frame Breadcrumbs* Scroll-spy Output/Terminal Window Status Bar	Integrated Development Environment (IDE) Text Editors Static Code Analysis Code Refactoring Debugging Tools	
15	<u>Shortcuts</u> (Ctrl + K Ctrl + S) <u>File</u> New (Ctrl + N) Open (Ctrl + O) Save (Ctrl + S) Save As (Ctrl + Shift + S) Folder (Ctrl + K Ctrl + O) Rename (F2) <u>Edit</u> Select All (Ctrl + A) Undo (Ctrl + Z) Redo (Ctrl + Y) Cut (Ctrl + X) Copy (Ctrl + C) Paste (Ctrl + V) Toggle Line Comment (Ctrl + /) Toggle Block Comment (Shift + Alt + A) Highlight (Double Click) Quote (Highlight and Shift + 2) <u>View</u> Toggle Navigation Pane (Ctrl + B) Explorer (Ctrl + Shift + E) Search (Ctrl + Shift + F) Split (Ctrl + \) View Directory (Shift + Alt + R) Close (Ctrl + F4) Close All (Ctrl + K Ctrl + W)		

	<u>Go</u> Back (Alt + LEFT) Forward (Alt + RIGHT) Next (Ctrl + PgUp) Previous (Ctrl + PgDown) Switch (Ctrl + TAB, Ctrl + Shift + TAB) History (Ctrl + P) Source Line (Ctrl + G) <u>Manage</u> Settings (Ctrl + ,) Themes (Ctrl + K Ctrl + T)		
16	<u>Terminal</u> (Ctrl + Shift + `) Root (cd\ View (dir) Navigate (TAB / Shift + TAB) Change (cd path\ Previous (cd.. Execute (ENTER) History (F7) Escape (ESC) Help (help) Clear (cls) Commands (cmd /?) Hidden (attrib -s -h -r /s /d) Close (Alt + F4)	Windows Command Line (cmd)	

MODULE 3: Advanced Data Structures with JavaScript Object Notation (JSON)			
#	LECTURE	SCOPE	EXERCISE
17	Introduction to JSON	Advanced Data Structures Database Schema	assembly.asm maven.json spreadsheet.json references.json

MODULE 4: Parallel Processing with Asynchronous JavaScript and XML (AJAX)

#	LECTURE	SCOPE	EXERCISE
18	Introduction to AJAX	Applications of Parallel Processing Threading Hypertext Transfer Protocol (HTTP) Web Services Simple Object Access Protocol (SOAP) RDF Site Summary (RSS) Search Engine Optimization (SEO) Instant Messaging (IM)	scanner.js *maven.json
19	Goto Statements	Unconditional Branching Basic Blocks	goto.js goto_rev_2.js
20	Pointers	Compiler Design Language Translators Intermediate Code Generation (Indirect Triples)	triples.js
21	Process Scheduling Scheduling Algorithms First Come, First Serve (FCFS) Shortest Job First (SJF) Round-robin	CPU Scheduling Criteria Time Complexity	fcfs.js sjf.js robin.js
22	Lexical Analysis ? Requirements traditionally imposed on Code Generators	Target Machine Code Lexical and Syntax Analysis Intermediate Code Generation (Quadruple)	quadruple.js *assembly.asm tokenizer.js
23	Instruction Cost ? Cost of a dot-product program	Memory Management Register Allocation	dot_prod_1.js dot_prod_2.js
24	Code Optimization ? Principal sources of Code Optimization	Peephole Optimization Memory Reallocation Overloaded Variable Declaration* Overloaded Variable Initialization* Parameterized Nesting* Lambda Return*	peephole.js
25	Prefix, Infix and Postfix Notation	List Programming (LISP)	lisp.js

MODULE 5: Server-side Scripting with Node.js Run-time Environment			
#	LECTURE	SCOPE	EXERCISE
26	Introduction to Server-side Scripting	Hypertext Transfer Protocol (HTTP) Common Gateway Interface (CGI)	
27	File Upload	File Transfer Protocol (FTP) Data Encryption	upload.js
28	File System	Folder Management System	stream.js
29	Email System	Computer Communication System Multipurpose Internet Mail Extensions (MIME)	email.js
30	Sessions	Computer Network Security	sessions.js

MODULE 6: Document-oriented Database Management with MongoDB

#	LECTURE	SCOPE	EXERCISE
31	Database Table Rows Fields Relationship Cardinality	Database Management System (DBMS) Entity Relationship Diagram (ERD) Persistent Storage	schema.js
32	<u>Basic SQL Commands</u> Create (INSERT INTO) Read (SELECT FROM) Update (UPDATE...SET) Delete (DELETE FROM)	Structured Query Language (SQL) Data Distribution Service (DDS) Representational State Transfer (REST)	crud.js
33	<u>Advanced SQL Commands</u> Search (WHERE...AND/OR) Sort (ORDER BY: ASC, DESC) Filter (DISTINCT, LIMIT)	Searching Algorithms Sorting Algorithms	search.js sort.js filter.js

MODULE 7: Software Development Life Cycle (SDLC) using Waterfall Model and MVC

#	LECTURE	SCOPE	EXERCISE
34	Requirement Phase	Software Development Life Cycle (SDLC) Systems Analysis & Design Facts Finding Techniques Cost Estimation Models	waterfall.doc cocomo.js
35	Design Phase	Business Process Modelling (BPM) Unified Modelling Language (UML) Entity Relationship Diagrams (ERD)	usecase.doc
36	Implementation Phase (Development)	Class and Object Diagrams	spreadsheet.js *spreadsheet.json
37	Testing Phase	Software Testing Life Cycle (STLC)	test_suite.js *spreadsheet.js
38	Deployment Phase	Secure Hypertext Transfer Protocol (HTTPS) Uniform Resource Locator (URL) Website Domain and Hosting Domain Name and Extension Internet Service Providers (ISPs) Web Servers and Cloud Storage ... Change Over Methods	
39	Maintenance Phase ? Why do systems change	Software Configuration Management Version Control Systems (VCS) Distributed Revision Control Systems (DRCS) Git Repository	
40	Software Documentation	Requirements Documentation Architecture Design Documentation Technical Documentation End-User Documentation Marketing Documentation	documentation.doc

MODULE 8: Software Prototyping with Hypertext Markup Language (HTML5)

#	LECTURE	SCOPE	EXERCISE
41	Introduction to Web Design Web Design Glossary Classification of Websites Careers in Web Design & Development	Website Architecture Content Management System (CMS)	
42	HTML5 Program Structure Tags Attributes Elements Meta Site Folder	Semantic Web MVC Architecture Symfony Folder Structure	index.html
43	Turbo Tags Basic Formatting Tags Hyperlinks Style Attribute Special Characters	Cognitive Science Case-based Reasoning (CBR) ... Literature Review (References) American Psychological Association (APA) American Medical Association (AMA) Council of Science Editors (CSE) ... CSS Display Property	references.html
44	Window Location Navigator Threading	Browser Object Model (BOM)	resize.html print.html countdown.html loading.html
45	Events Event Listeners Event Handlers ? The impact of <i>Gamification</i> in E-Learning	Document Object Model (DOM) Rich Text Editors ... Game Development Interactive Prototyping Bouncing Ball Algorithm	onclick.html onchange.html onkeyup.html bison.html*
46	Heading Paragraphs and Line Breaks Image Footer Address Time		index.html
47	Forms Form Validation API		login.html register.html

	HTTP POST Sessions		stocks_add.html sales_add.html settings.html
48	Header Nav Main HTTP GET Lists	Enterprise Resource Planning (ERP)	dashboard.html sales_preview.html
49	Tables	Inventory Management System Business Intelligence	stocks_record.html sales_record.html sales_report.html accounts.html
50	Article		about.html
APX	Monetization <u>Understanding Business Models</u> Market Research Consumerbase Consumer Metrics Consumer Satisfaction Brand Perception <u>Principles of BLEND Design</u> User Interface (UI) User Experience (UX) User Engagement (UE) <u>Revenue Markup Language (RXML)</u> Advert Placement Sponsored Content Classifieds Model E-Commerce Network Marketing Subscription Service Rental or Leasing Model ‘Freemium’ Model Drop Shipping ? Business Model v Revenue Model ? User-base v Consumer-base ? The Real Product-Market Fit	<u>Subscription Service Models</u> Software as a Service (SaaS) Platform as a Service (PaaS) Infrastructure as a Service (IaaS) Backend as a Service (BaaS) <u>Consumer Metrics</u> Organic Growth Monthly Active Users (MAU) Session Rate Conversion Rate Retention Rate <u>Marketing Mix (7P’s)</u> Product Price Place Promotion People Process Physical Environment <u>SWOT Analysis</u> Strengths Weaknesses Opportunities Threats	

EXAMINATION & CERTIFICATION

FOREWORD

Two core **attributes** of a Maven programmer are Confidence and Perseverance. A Maven programmer exhibits certain **behaviours** when writing code; Algorithm Design, Semantic Nomenclature, Identifier Hoisting, Code Readability (tidy) and Code Optimization. Thus, the 3 Ps of programming are Practice! Practice!! Practice!!!

...

“use strict”;

class Maven

{

 constructor (agr1, agr2)

 {

this.confidence = agr1;

this.perseverance = agr2;

 }

get algorithmDesign() {**return** true;}

get semanticNomenclature() {**return** true;}

get identifierHoisting() {**return** true;}

set codeReadability (tidy) {**window.tidy** = true;}

get codeOptimization() {**return** true;}

thus = () => {**for** (**let** i = 1; i <= 3; i++) **alert**(“Practice!!!”);}

}

// TODO Express the illustration above using object-oriented programming

const programmer = **new** Maven(true,true);

programmer.thus();

REFERENCES

- Brendan Eich, creator of JavaScript scripting language 1995, Netscape Communications Corporation, Mozilla Foundation, ECMA International, <https://www.javascript.com/>.
- Douglas Crockford, author of JavaScript Object Notation (JSON) 2000, Paypal, <https://www.json.org/>.
- Emmanuel Tugbeh, author of SlingshotJS framework for Masters in Computer Science 2019, SlingshotJS Working Group, HWP Labs, <http://www.hwplabs.com/>.
- Google LLC., owner of Google Chrome 2008, <https://www.google.com/chrome/>. Creator of Asynchronous JavaScript and XML (AJAX) 2004, coined by in Jesse James Garrett in 2005, standardized by the World Wide Web Consortium (W3C) in 2004, <https://www.w3c.org/>.
- Herbert Benington, author of Waterfall SDLC Model 1956, Symposium on Advanced Programming Methods for Digital Computers, <https://www.worldcat.org/oclc/10794738/>.
- Isaac Schlueter, developer of Node Package Manager (NPM) 2010, Joyent Inc., <https://www.npmjs.com/>.
- Microsoft Corporation, owner of Visual Studio Code Integrated Development Environment (IDE) 2015, <https://code.visualstudio.com/>.
- Ryan Dahl, creator of the Node.js JavaScript run-time environment 2009, Joyent Inc., <https://www.nodejs.org/>.
- Tim Berners-Lee, developer of Hypertext Markup Language (HTML) 1993, Web Hypertext Application Technology Working Group (WHATWG), <https://www.whatwg.org/>.
- Trygve Reenskaug, author of Model-View-Controller (MVC) 1970, Xerox Palo Alto Research Center (PARC), Microsoft Corporation, <https://dotnet.microsoft.com/apps/mvc/>.